

Composite Entity-Addressed Keys for CRDT Maps

Merkle, B.¹ Rodionov, V.²

Draft — preprint, April 2026

Abstract

CRDT maps restrict keys to strings — a constraint inherited from JSON, not from CRDT algebra. We present a key serialization algebra that embeds primitives, entity references, ordered tuples, and unordered sets into string keys with deterministic cross-peer serialization. Set keys are sorted after serialization, making $\{A, B\}$ and $\{B, A\}$ identical across peers. With self-resolving entity identity (§5 of^[1]), entity references in keys are decodable back to the CRDT operation log without a lookup table. The technique enables CRDT-native composite key addressing: junction-free relations and multi-dimensional lookups expressed as single map entries, without application-managed indexes. Decodable entity references additionally enable referential GC without a reverse index (P6).

1 The String Key Constraint

Every major CRDT map implementation restricts keys to strings:

Framework	Key type	Source
Yjs Y.Map	<code>string</code>	Hard-coded in API ^[2]
Automerge	<code>string</code>	JSON object model ^[3]
Loro	<code>string</code>	Hard-coded in API ^[4]

This restriction inherits from the JSON data model: JSON object keys are strings. CRDT frameworks that model “JSON-like documents” inherit the constraint. Notably, the CRDT algebra itself imposes no such restriction — standard formalizations of OR-Map and LWW-Map parameterize over a key set K with no structural requirement beyond equality^[5,6]. The string constraint is a convention, not a necessity.

The one production exception is Riak DT Map^[7], which uses composite keys (`name: binary, type: atom`) where `type` is one of `{counter, set, register, flag, map}` — a fixed schema-level discriminator that distinguishes “score as counter” from “score as set,” not an arbitrary composite key.

1.1 What String Keys Cannot Express

Entity-keyed lookups. A design tool stores style overrides per component. The natural key is the component entity, but with string keys the application must serialize the entity’s ID, handle key stability, and resolve keys back to entities via a lookup table or

secondary index. Composite relationships compound this: a collaborative spreadsheet applies conditional formatting rules per cell, and the natural key is the tuple (`row`, `column`, `rule`) — three entity references. String keys require an ad hoc serialization convention (“`{rowId}::{colId}::{ruleId}`”) that every consumer must implement identically.

Unordered set keys. A visual component has style settings per *combination* of active variants — what does it look like when both `hover` and `disabled` are active? The natural key is the unordered set `{hover, disabled}`. String keys require sorting entity IDs, choosing a delimiter, and ensuring all peers use the same convention; the industry workaround is exactly this: `JSON.stringify(combo.map(v => v.uuid).sort())`. A closely related case is many-to-many relationships, which in relational databases use junction tables with composite primary keys; in CRDTs this requires either a junction entity (doubling the operation count) or a manually-serialized composite string key.

The universal pattern across frameworks is `Map<string_serialized_id, Data>` with secondary indexes built outside the CRDT^[3]. Loro explicitly advises against using CRDTs for “*global invariants (referential integrity, cross-document totals)*”^[4]. The technique here does not enforce global invariants — it expresses structural relationships that converge by construction, which is a different concern.

2 Technique

The key serialization algebra is a layer above the CRDT map, not a modification to it. Composite keys serialize to plain strings and store in any existing string-keyed

¹Independent researcher, Here.build; x.com/merkle_bonsai; ImmuneFi top-40 whitehat (ethical hacker); email: merklebonsai@pm.me

²Independent researcher, Here.build; email: v@here.build

CRDT map. The CRDT’s conflict resolution semantics are unchanged.

2.1 Key Type Algebra

Three constructors over a value domain:

```

Key ::= Value(v)           -- single value
      | Array(v1, v2, ..., vn) -- ordered
      composite (n >= 0)
      | Set(v1, v2, ..., vn) -- unordered
      composite (n >= 0, no duplicates)

v ::= string | number | boolean | bigint | null |
    EntityRef(id)

```

`EntityRef(id)` is a reference to a CRDT entity, where `id` is the entity’s self-resolving identifier (§5 of^[1]) — a 15-character string encoding the creating operation’s `{clientId, clock}` pair. Nesting is not supported: keys cannot contain other keys as elements (see §2.2).

The constructors form a tagged union. The type tag (`Value`, `Array`, `Set`) is part of the serialized form, ensuring that `Value(42)` and `Array(42)` are distinct keys.

Empty containers are valid: `Set()` represents “no entities” (e.g., base variant with no active states), `Array()` represents an empty tuple.

2.2 Serialization

Keys serialize to newline-delimited strings with a type prefix:

```

Value key:  "Value\n" + serializeValue(v)
Array key:  "Array\n" + serializeValue(v1) + "\n" +
            ... + "\n" + serializeValue(vn)
Set key:    "Set\n" + sort([serializeValue(v1), ...,
                           serializeValue(vn)]).join("\n")

```

Value serialization. Primitives serialize via `JSON.stringify`, which escapes newlines to `\n` — ensuring the newline delimiter is unambiguous. Special cases handled explicitly: `NaN` → `"NaN"`, `Infinity` → `"Infinity"`, `-Infinity` → `"-Infinity"`, bigints via trailing `n` suffix (`42n`). Entity references serialize to their CRDT reference tuple: `["pRxK4mN7pQ2wJbL"]` for local entities, `["pRxK4mN7pQ2wJbL", "dep123"]` for cross-document references.

Set commutativity. Set elements are serialized individually, then sorted lexicographically as strings. This produces identical key strings regardless of insertion order, without requiring a total order on the heterogeneous value domain. The sort operates on

serialized forms (byte strings), which always have a total order.

Injectivity. The serialization is injective (distinct keys produce distinct strings) under the following conditions:

- (a) `JSON.stringify` is injective for the supported primitive types — which holds except for `-0` vs `0` (both produce `"0"`) and numbers beyond `Number.MAX_SAFE_INTEGER` (precision loss). These edge cases should be validated at the serialization boundary.
- (b) Entity identifiers are unique per the CRDT’s own guarantee (§5 of^[1]).
- (c) The type prefix and newline delimiter are unambiguous because all value forms JSON-escape literal newlines. Literal backslashes in string payloads are themselves JSON-escaped (`"\\n"` encodes the two characters `\n`), so the delimiter escape is unambiguous at all depths of backslash nesting.
- (d) Empty containers across constructors are distinguishable by type prefix alone: `Array()` serializes to `"Array"` (no trailing elements), `Set()` to `"Set"`, `Value()` requires a value (empty not supported). No two empty container prefixes collide. Empty string values serialize unambiguously: `JSON.stringify("")` is `'"'`, which does not prefix-collide with any non-empty value.
- (e) Set sort stability across implementations requires attention. The sort is over serialized byte strings — total order by construction — but the *specific* order depends on the sort’s collation. JavaScript’s default `Array.prototype.sort()` uses UTF-16 code-unit order; Rust’s and Go’s default string sort uses UTF-8 byte order. For ASCII-only serialized forms (primitives, entity references, genesis paths in `[a-zA-Z0-9_]+`) the two coincide. For Set elements containing non-ASCII string values (or surrogate-pair code points), JS and Rust sorts diverge. Mitigations, in order of preference: (i) normalize at the serialization layer — NFC-normalize strings and require ASCII-only Set element forms via percent-escaping or equivalent; (ii) re-encode to UTF-16 inside the sort comparator on non-JS runtimes (correct but expensive per-comparison); (iii) document Set-key payloads as ASCII-only. This is a portability constraint on P3.

Nesting prohibition. Composite keys cannot contain other composite keys as elements. The flat

newline-delimited format cannot parse nested containers unambiguously without adding an escaping or depth mechanism, which would complicate both injectivity analysis and framework compatibility. Applications needing nested structure should compose via entity references — the nested structure lives as an entity, and the key holds the reference.

2.3 Resolution

Deserialization splits on newlines, reads the type prefix, and resolves each element. Entity references are decoded from the self-resolving identifier to `{clientId, clock}`, then resolved via binary search on the CRDT operation log (§5 of^[1]). Resolution is $O(\log m)$ per entity reference, where m is the operation count for the relevant client — with no auxiliary index or lookup table.

3 Properties

3.1 Standalone (no companion dependencies)

P1. Cross-peer determinism. Two peers serializing the same key produce the same string. Set keys are order-independent by construction (sort-after-serialize). This holds for any CRDT map that accepts string keys.

P2. Collision-free under supported types. Distinct keys produce distinct serialized strings, given: supported primitive types (excluding `-0` and unsafe integers), unique entity identifiers, and the tagged-union type prefix.

P3. Framework-compatible. The serialized key is a plain string. It stores in Yjs `Y.Map`, Automerge maps, Loro maps, or any string-keyed CRDT map without protocol changes. Frameworks with aggressive key interning or columnar compression may see disproportionate performance impact from long composite keys.

3.2 Composed (with self-resolving entity identity, §5 of^[1])

P4. Decodable keys. With self-resolving entity identity (§5 of^[1]), every entity reference in a key can be resolved to a live entity via decode + binary search on the CRDT operation log. No lookup table or secondary index required. A serialized key received over any channel (network, clipboard, snapshot) is self-contained — every entity reference is a live pointer.

P5. Stable entity references. Entity identifiers are derived from the creating operation’s coordinates (§5

of^[1]) and never change. A key referencing entity A remains valid regardless of how A’s content evolves. This is structurally stronger than using mutable properties (names, paths) as key components.

P6. Decodable keys enable referential GC without a reverse index. Because every entity reference in every map key is decodable directly from the key string, garbage collection of deleted entities can scan map keys for references to the deleted entity and cascade-delete the affected entries — without maintaining a reverse reference index. The scan is $O(\text{map size})$ worst case per deletion, which bounds applicability; for large high-cardinality maps a reverse index may still be preferable. But the option *exists*, which is not true for CRDT maps using opaque string keys: there, the only way to find entries referencing a deleted entity is to deserialize every key on every deletion anyway, or to maintain an external index. Decodability makes the index optional rather than mandatory. This is a key-level scan, complementary to the whole-graph reachability signal of §4 of^[1] — the architecture paper’s reachability walks the reference graph from root via schema-declared edges; P6 here is local to a single map’s keys and operates without reference to the graph walk.

Note: when combined with deterministic genesis^[8] and replica-identifier partitioning^[2] (itself an adaptation of Yu et al.’s categorical-second-dimension technique to op-based CRDTs), structural entities referenced in keys gain additional properties — convergent creation and priority ordering — but these are contributions of the companion papers, not of the key algebra itself.

4 Applications

4.1 Entity-Keyed Lookups

The simplest case — one entity as a map key:

```
Map<Component, StyleOverride>
```

The component’s self-resolving identifier serves as the key. Stable across peers, survives serialization, resolves without a registry.

4.2 Composite Relationships (Tuples)

Two entities jointly own a configuration:

```
Map<[Component, Slot], Binding>
```

The array key is an ordered composite — $[A, B] \neq [B, A]$. No junction entity is needed; the key carries the relation directly.

4.3 Multi-Dimensional Lookups (Sets)

A visual component has style settings per variant combination:

```
Map<Set<Variant>, StyleSettings>
```

The set key `{hover, disabled}` serializes identically regardless of insertion order. This eliminates the ad hoc `JSON.stringify(combo.map(v => v.uuid).sort())` pattern found in production Yjs applications. Two peers writing style settings for the same variant combination address the same map entry — their writes merge correctly via the underlying CRDT’s per-key conflict resolution.

Note: set-keyed maps provide exact-match lookups. Range queries (“find all entries whose key is a subset of this set”) require additional application logic or a secondary index.

4.4 Hyperedges

A hyperedge connects N nodes. The edge identity is the set of connected nodes:

```
Map<Set<Node>, EdgeData>
```

No separate edge entity is required: the set of nodes serves as the edge’s address, so two peers creating an edge between the same nodes produce the same set key and the edge converges without coordination.

5 Constraints

C1. Dangling references. When a referenced entity is deleted, keys containing it become partially unresolvable. This mirrors the dangling foreign key problem in relational databases. P6 describes the referential-GC option that decodability enables; C4 enumerates the specific interaction with CRDT runtime GC.

C2. Set key cardinality. A set of N entities produces a key string of approximately $N \times 17$ characters (15-char identifier + delimiters). CRDT maps store the full key string per entry. High-cardinality sets (100+ elements) inflate storage and sync payloads. A content-addressed hash of the sorted set would be more compact at the cost of losing decodability.

C3. Primitive edge cases. -0 and 0 serialize identically via `JSON.stringify`. Numbers beyond

`Number.MAX_SAFE_INTEGER` lose precision during serialization. The serialization boundary should validate these cases. BigInt serialization uses an unquoted `n` suffix, distinguished from string `"42n"` by the JSON quotes around the string form.

C4. Garbage collection interaction. Referenced entities must be retained for the key to remain resolvable. Three interaction modes exist: (a) `gc: false` — everything works at the cost of unbounded tombstone growth; (b) GC with reference scanning via P6’s key-scan approach — acceptable for small maps and periodic cleanup but non-trivial at high cardinality; (c) GC without reference scanning — dangling keys are inevitable; applications must tolerate partial resolution. The cold-storage offload alternative of §4 of^[1] generalizes (b) to tiered storage: unreachable entities move to a cold tier rather than being deleted, and their keyed references resolve via cold-tier fetch. This replaces the scan cost with a per-offload cost that amortizes well.

C5. Portability. The key serialization algebra (P1-P3) works with any CRDT framework using its native entity identifiers — Automerge objectIds (`"{counter}@{actorId}"`), Loro TreeIds, or application-generated UUIDs. With Automerge objectIds, entity-keyed lookups already work (objectIds are resolvable via `doc.getObjectById()`). The decodability property (P4) specifically requires self-resolving identifiers (§5 of^[1]) but the algebra itself does not.

6 Related Work

Riak DT Map^[7] is the only CRDT with *compound structured keys* in production — most production CRDT maps use string keys, and applications compose into those strings ad hoc. Our specific contribution is a structured encoding of *decodable entity references* inside composite keys: the novelty is not composite keys per se (the industry workaround `JSON.stringify(combo.map(v => v.uuid).sort())` is essentially the scalar case), but rather (a) a type-prefixed injective serialization that includes entity references as first-class elements, (b) the Set-key commutativity via sort-after-serialize, and (c) the referential-GC consequence (P6) that decodability enables. Riak DT Map’s fixed schema discriminator does not compose with multi-entity references or set keys.

Kleppmann and Beresford^[9] formalize JSON CRDTs with string-keyed maps, establishing the convention this paper extends. For LWW-Map the serialized string is the LWW unit; for OR-Map it is

the per-key tombstone unit. Convergence semantics of the underlying map are preserved regardless of the host CRDT’s map variant.

Baquero, Almeida, and Shoker^[5] define pure operation-based CRDTs with generic key types, demonstrating that the CRDT algebra does not require string keys. Our technique bridges this theoretical generality with the practical string-key constraint of deployed frameworks.

SynQL^[10] replicates relational foreign keys as string attributes — the “serialize entity ID to string” pattern. It does not support composite or set-valued keys.

Shapiro et al.^[11] prove that causal consistency suffices for referential integrity. Our entity references in keys are not enforced by the CRDT — they are structural relationships expressed through serialization, not integrity constraints.

Weidner’s CRDT-Valued Map^[12] formalizes lazy maps where every key implicitly has a value. With set keys, the logical keyspace is the powerset of the entity space — every possible combination of entities becomes a valid key.

Adjacent patterns outside CRDT research. RDF triple/quad stores^[13] model data as (**subject**, **predicate**, **object**) tuples — composite keys with entity references as first-class components. Production RDF stores (Apache Jena, Oxigraph) serialize these tuples into string-keyed B-trees using pattern-specific encodings; our algebra generalizes this to arbitrary tuple/set shapes with a single serialization rule. Non-CRDT production systems such as Facebook’s TAO and Datomic materialize similar (**entity**, **attribute**, **value**) composite structure in non-replicated settings; the contribution here is making this structure work under CRDT convergence. Yjs **RelativePosition**^[14] and Loro **ContainerID** serialize coordinate-derived identifiers into strings that are decodable into CRDT positions — the structural cousin of our EntityRef, but scoped to position-within-sequence rather than entity-within-graph. TreeDoc / RGA position identifiers^[15] likewise embed decodable structure into sort keys; the use case (list ordering) differs but the encoding pattern — “serialize a coordinate tuple into a totally-orderable string” — is the same.

Entity-oriented CRDT documents^[1] provide the architectural setting: a document is a flat registry of entity shells connected by a reference graph, with a concrete self-resolving UUID encoding (§5 of^[1]) that makes references decodable, cross-peer-stable, and usable without a lookup table. Keys with EntityRef components are a natural extension of that graph into

map-key positions. Deterministic genesis^[8] and replica-identifier partitioning^[2] — the latter an adaptation of Yu et al.’s categorical-second-dimension technique to op-based CRDTs — provide additional guarantees (convergent creation, priority ordering) for structural entities but are not required by the key algebra itself.

7 Conclusion

CRDT maps have been string-keyed since their formalization. The CRDT algebra imposes no such restriction — it is inherited from the JSON data model. We provide a structured serialization that embeds richer key types into strings while preserving the properties needed for correct CRDT convergence.

The core contribution — a key type algebra with entity references, ordered tuples, and unordered sets — is adoptable by any CRDT framework with string-keyed maps. When it is composed with self-resolving entity identity (§5 of^[1]), keys become decodable pointers: every entity reference in a key resolves directly to the CRDT operation log, and referential GC (P6) becomes tractable without a reverse reference index. Applications built on this composition no longer need junction entities, ad hoc key serialization conventions, or the assumption that map keys are opaque strings.

The algebra itself is CRDT-external. P4 and P6, however, depend on the serialized key being able to round-trip into the host CRDT’s operation-log coordinates — a property that requires coordination between the key schema and the host’s identity model. Opaque-string alternatives cannot deliver this, which is why the contribution is better described as a serialization convention that integrates with a coordinate-addressed CRDT identity layer than as a standalone string-encoding scheme.

References

- [1] Plexus Project. Entity-oriented CRDT documents: Architecture, identity, and structural liveness. Preprint, 2026.
- [2] Plexus Project. Adapting categorical CRDT lifecycle to op-based CRDTs via replica-identifier partitioning. Preprint, 2026.
- [3] Automerge. Modeling data. <https://automerge.org/docs/cookbook/modeling-data/>, 2024.
- [4] Loro. When not to use CRDTs. https://loro.dev/docs/concepts/when_not_crdt, 2024.

- [5] Carlos Baquero, Paulo Sérgio Almeida, and Ali Shoker. Pure operation-based replicated data types. *arXiv:1710.04469*, 2017.
- [6] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. *INRIA RR-7506*, 2011.
- [7] Russell Brown, Sean Cribbs, Christopher Meiklejohn, and Sam Elliott. Riak DT map: a composable, convergent replicated dictionary. In *Proc. PaPEC (ACM)*, 2014.
- [8] Plexus Project. Deterministic genesis: Coordination-free structural initialization for operation-based CRDTs. Preprint, 2026.
- [9] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE TPDS*, 2017.
- [10] Claudia-Lavinia Ignat, Victorien Elvinger, and Maëva Ba. Synql: A CRDT-Based approach for replicated relational databases with integrity constraints. In *Proc. DAIS*, 2024.
- [11] Marc Shapiro, Annette Bieniusa, Peter Zeller, and Gustavo Petri. Ensuring referential integrity under causal consistency. 2018.
- [12] Matthew Weidner. Designing data structures for collaborative apps. <https://mattweidner.com>, 2022.
- [13] W3C. RDF 1.1 concepts and abstract syntax. <https://w3.org/TR/rdf11-concepts/>, 2014.
- [14] Yjs Authors. Yjs RelativePosition. <https://docs.yjs.dev/api/relative-positions>, 2024.
- [15] Nuno Preguiça, Joan Manuel Marquès, Marc Shapiro, and Michaël Letia. A commutative replicated data type for cooperative editing. In *Proc. IEEE ICDCS (Treedoc)*, 2009.